

Scribed by:

1. Devesh Yadav (2022MT11281)
2. Niraj Agarwal (2022MT11921)
3. Arnav Singhal (2022MT11270)
4. Rohit Shaw (2022MT61988)
5. Anubhav Karadia (2022MT11292)

1 Overview of Turing Machine Concepts**Language Classes and Decidability**

A Turing Machine (TM) can either recognize or decide a language.

Turing-Recognizable A language L is recognizable if a TM M halts and accepts for every string $w \in L$. For $w \notin L$, M may reject or loop forever.

Turing-Decidable A language L is decidable if a TM M (a **decider**) is guaranteed to halt on all inputs, always returning a definitive accept or reject.

Equivalence of TM Models

Different models of Turing machines are equivalent in computational power.

- **TM with a "Stay" Option:** A TM with moves $\{L, R, S\}$ is equivalent to a standard TM with moves $\{L, R\}$, as a "Stay" move can be simulated by one move Right followed by one move Left.
- **Multi-tape TM:** A TM with k tapes is equivalent to a single-tape TM. The simulation involves storing all k tapes on a single tape, separated by a special symbol $\#$, and using marked symbols (e.g., $\dot{a}$) to track the k virtual head positions.

Nondeterministic Turing Machines (NTMs)

An NTM's transition function specifies a set of possible next moves, allowing it to explore multiple computation paths.

Definition (NTM Transition Function).

$$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

An NTM accepts an input if **at least one** path in its computation tree leads to an accept state.

Theorem. *Every Nondeterministic Turing Machine has an equivalent Deterministic Turing Machine.*

Proof Sketch. A DTM can simulate an NTM by performing a **breadth-first search (BFS)** of the NTM's computation tree. This is often described using a 3-tape DTM: one tape for the read-only input, one for simulating a path, and one "address" tape to systematically guide the exploration of all paths. $\square$

Now in this lecture we will discuss

- **Equivalence of DTMs and NTMs:** Deterministic (DTM) and Nondeterministic (NTM) Turing Machines are equivalent in computational power. A DTM can simulate an NTM by performing a breadth-first search of the NTM's computation tree, which guarantees finding a finite accepting path if one exists.
- **Graph Connectivity Algorithm:** A Turing machine can decide if a graph is connected. The algorithm involves representing the graph on the tape (e.g., as an adjacency list) and performing a traversal (like BFS) by marking visited vertices. If all vertices are marked after the traversal, the graph is connected.
- **The Church-Turing Thesis:** This is a fundamental principle stating that any function that can be computed by an effective algorithm can also be computed by a Turing machine. It formally links the intuitive concept of an "algorithm" to the mathematical model of a Turing machine.

2 Nondeterministic Turing Machine

A Nondeterministic Turing Machine (NTM) is a Turing machine where the transition function δ allows for a set of possible outcomes for each state-symbol pair. The function is defined as:

$$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

where $\mathcal{P}(\cdot)$ denotes the power set. An NTM accepts an input string w if **at least one** of its possible computation paths leads to an accept state.

Theorem. *Every Nondeterministic Turing Machine has an equivalent Deterministic Turing Machine.*

Proof by Simulation. We prove this by constructing a Deterministic Turing Machine (DTM), let's call it D , that simulates a given NTM, N . To avoid getting trapped in an infinite computation branch, D performs a **breadth-first search (BFS)** of N 's computation tree.

We describe D as a 3-tape DTM for clarity:

Tape 1 (Input Tape): Holds the original input string w . This tape is never altered during the simulation process.

Tape 2 (Simulation Tape): A work tape. Before simulating each path, the original input w is copied here from Tape 1.

Tape 3 (Address Tape): Stores a string representing a path through N 's computation tree. For example, if the maximum branching factor is b , this tape will hold a sequence of digits from 1 to b . The string "213" instructs the simulation to take branch 2 at the first choice, branch 1 at the second, and so on.

The DTM D executes the following algorithm:

1. **Initialization:** The input string w is on Tape 1. Tapes 2 and 3 are empty.

2. **Loop Start:** Generate the next path address string on Tape 3 in lexicographical order (e.g., "1", "2", ..., "b", "11", "12", ...).
3. **Setup Simulation:** Copy the input w from Tape 1 to Tape 2. Reset the head on Tape 3 to the beginning of the address string.
4. **Simulate Path:** Run the computation of N on Tape 2. Whenever N has a nondeterministic choice, consult the next digit on Tape 3 to decide which branch to take.
5. **Check Outcome:**
 - If the simulation reaches an **accept state**, then D halts and **accepts**.
 - If the simulation reaches a **reject state**, or if the path is invalid (e.g., the address on Tape 3 specifies a branch that doesn't exist), abandon this path and go back to Step 2 to try the next one.
 - If the end of the address string on Tape 3 is reached before the simulation halts, abandon this path (as it's a deeper path to be explored later) and go back to Step 2.

This breadth-first search strategy ensures that D explores all computation paths of length 1, then all paths of length 2, and so on. If the NTM N has an accepting path of any finite length, D is guaranteed to eventually find it, simulate it, and accept. Therefore, $L(D) = L(N)$. $\square$

Theorem. *A language L is recognizable by a Deterministic Turing Machine (DTM) if and only if it is recognizable by a Nondeterministic Turing Machine (NTM).*

Proof. The proof consists of two parts.

Part 1: If L is recognizable by a DTM, it is recognizable by an NTM. This direction is straightforward. Any DTM is a special case of an NTM where the transition function, for every state and symbol, yields a set containing exactly one possible move. Therefore, a DTM is an NTM that does not utilize nondeterminism.

Part 2: If L is recognizable by an NTM, it is recognizable by a DTM. We prove this by constructing a DTM, D , that simulates a given NTM, N . The DTM D will perform a breadth-first search (BFS) of N 's computation tree to ensure it finds an accepting path if one exists, avoiding infinite loops that could trap a depth-first search.

We can describe D as a 3-tape DTM:

Tape 1 (Input Tape): Holds the original input string w and is never altered.

Tape 2 (Simulation Tape): A work tape where D simulates one branch of N 's computation.

Tape 3 (Address Tape): Stores a sequence of numbers representing a path in N 's computation tree. Each number directs which choice to make at a nondeterministic step.

The DTM D operates by repeatedly generating the next address string on Tape 3 in lexicographical order (thus exploring paths in a BFS manner). For each address, it copies the input from Tape 1 to Tape 2 and simulates that specific path. If the simulation on Tape 2 reaches an accept state, D halts and accepts. Otherwise, it proceeds to the next address.

This systematic search guarantees that D will find any finite accepting path of N . Thus, $L(D) = L(N)$, and the two machine types are equivalent in power. $\square$

3 Problem: Graph Connectivity on a Turing Machine

Here we outline an algorithm for a Turing machine to determine if a given undirected graph G is connected.

1. Encoding the Graph. First, the graph must be represented on the TM's tape. A common encoding is an adjacency list. For a graph with vertices $\{1, 2, \dots, n\}$, the tape could be formatted as:

$$(1 : v_{1,1}, v_{1,2}, \dots)(2 : v_{2,1}, \dots) \dots (n : \dots)$$

2. The Algorithm. The TM can use a graph traversal algorithm like Breadth-First Search (BFS).

1. **Mark the Start:** Find the first vertex on the tape and mark it as visited using a special symbol (e.g., a dot, changing '1' to '.').
2. **Iterative Traversal:** Repeatedly sweep through the tape. On each sweep, find a marked vertex 'u' and then find all of its neighbors 'v' from its adjacency list. If any neighbor 'v' is not yet marked, mark it.
3. **Termination:** Continue this process until a full sweep across the tape results in no new vertices being marked.
4. **Final Check:** After the traversal terminates, perform one final scan of all vertices.
 - If all vertices are marked, the graph is connected. The TM halts and **accepts**.
 - If any vertex is found to be unmarked, the graph is not connected. The TM halts and **rejects**.

4 The Church-Turing Thesis

The Intuitive Notion of an "Algorithm" Before the 1930s, an "algorithm" was an intuitive concept, understood as a mechanical, step-by-step procedure for solving a problem that required no special ingenuity to execute (e.g., long division). However, without a formal mathematical definition, it was impossible to prove that a problem *could not* be solved by any algorithm.

A Convergence of Formal Models In the 1930s, mathematicians independently developed formal systems to capture this idea of "effective computability." Remarkably, these different systems—including Alonzo Church's lambda calculus, Kurt Gödel's recursive functions, and Alan Turing's Turing machines—were all proven to be equivalent in computational power. This suggested they had all converged on the same fundamental concept.

Principle (The Church-Turing Thesis). *The thesis formalizes the connection between the intuitive and the mathematical. It claims that the formal power of a Turing machine is identical to the intuitive power of an "algorithm." It can be stated informally as:*

"Every Turing machine is an algorithm & every algorithm has a Turing machine."

The first part of this statement is true by definition; a TM is a precise, mechanical procedure. The second part is the core of the thesis: the assertion that any effective procedure we can conceive of can be simulated by a Turing machine.

Why It's a Thesis, Not a Theorem The Church-Turing Thesis cannot be formally proven because "effective algorithm" is an informal, philosophical concept. We cannot mathematically prove that an informal idea is equivalent to a formal definition. It is a hypothesis that is accepted based on overwhelming evidence, namely that no one has ever found a convincing counterexample.

Significance and Implications The thesis is a cornerstone of theoretical computer science. It provides a stable, formal definition of what is computable, allowing us to:

- **Define the Limits of Computation:** By proving that no Turing machine can solve a problem (like the Halting Problem), we can confidently assert that no algorithm of any kind can solve it.
- **Provide a Theoretical Foundation:** The modern computer is an implementation of the Universal Turing Machine concept. The thesis establishes the theoretical capabilities and limitations of these devices.

References

- [1] P. Linz, *An Introduction to Formal Languages and Automata*, 6th Edition, Jones & Bartlett Learning, 2016.